

Pothole Safety Subsystem Specification

Complete Technical Guide to Speed Management, Wheel-Track Geometry, and Classifier Inference

Metric / Scope	Specification / Target
Subsystem: Pothole & Surface Safety	Module Path: core_safar_logic/pothole_system
Target Speeds: 12.0 / 7.0 / 0.0 m/s	Braking Rates: 2.0 / 4.0 / 8.0 m/s^2
Wheel Straddle Clearance: 0.16m	Benchmark Pass Rate: 100% (12 / 12 Scenarios)

1. Subsystem Architecture & Physical Principles

The SAFAR Pothole Safety Subsystem provides autonomous detection, physical wheel-track strike evaluation, and graduated speed management for road surface depressions. It eliminates false-positive emergency braking by evaluating both lateral tire track geometry (left/right tire spans) and vehicle ground clearance (16cm). Potholes passing harmlessly between the wheels do not trigger braking, while severe craters spanning the tires execute full emergency deceleration.

2. Graduated Speed Policy & Deceleration Formula

Speed management is governed by exact target speed ceilings and continuous physical deceleration:

- **Class 0 (drivable_path):** Maintain driver speed (0.0 m/s^2 intervention).
- **Class 1 (Sml_ph):** Smooth slowdown to 12.0 m/s (43.2 km/h) at 2.0 m/s^2 .
- **Class 2 (Mid_ph):** Controlled crawl to 7.0 m/s (25.2 km/h) at 4.0 m/s^2 .
- **Class 3 (Crater):** Emergency braking at 8.0 m/s^2 down to complete stop (0.0 m/s).
- **Distance-Coupled Deceleration:** $a_{\text{req}} = \frac{v_{\text{ego}}^2 - v_{\text{target}}^2}{2 \cdot d_{\text{forward}}}$.

3. Source Code Files Breakdown

File Name	Responsibility & Architectural Work
speed_manager.py	Implements `PotholeSpeedManager` and `PotholeActionPlan`. Contains `manage_speed()` (exact 12/7/0 m/s policy), `evaluate_with_physics()` (distance deceleration coupling), and `check_wheel_strike()`.
road_simulator.py	Implements `simulate_road(road, starting_speed)` and standalone CLI. Steps through road nodes, classifies road depressions, checks wheel-track intersection, and outputs node-by-node telemetry.
classifier.py	Implements `PotholeClassifier` and `PotholeObservation`. Evaluates fast physical boundary rule (depth <= 8mm is drivable path) + pre-trained DecisionTree / GradientBoosting models on 3 features.
physics.py	Implements `PotholePhysicsEngine`. Computes vehicle stopping distance $d_{\text{stop}} = v \cdot t_{\text{react}} + \frac{v^2}{2a}$ and Time-To-Pothole ($TTP = d / v$).
path.py	Implements `PotholePathGeometry`. Evaluates tire track spans (Left: [-0.925, -0.675]m, Right: [+0.675, +0.925]m) and ground clearance (0.16m).
risk.py	Implements `PotholeRiskEngine` and `PotholeRiskAssessment`. Computes continuous severity scores and maps to SAFE, LOW, MEDIUM, HIGH, CRITICAL.
decision.py	Implements `PotholeDecisionEngine`. Hysteresis state machine enforcing 350ms active hold timer to prevent single-frame braking chatter.
validation.py	Implements `PotholeDataValidator`. Verifies dimension boundaries (width < 10m, length < 20m, depth < 1m) and rejects NaN/negative inputs.

4. Line-by-Line Code Walkthrough

```
class PotholeSpeedManager:
    TARGET_SPEEDS = {0: None, 1: 12.0, 2: 7.0, 3: 0.0}
    BRAKING_RATES = {0: 0.0, 1: 2.0, 2: 4.0, 3: 8.0}

    def manage_speed(self, current_speed: float, pothole_type: int) -> Tuple[float, str]:
        if current_speed <= 0.0: return 0.0, 'STOPPED waiting for driver to take action!!!'
        if pothole_type == 0: return current_speed, 'Allow the driver to adjust speed'
        elif pothole_type == 1:
            tgt = self.TARGET_SPEEDS[1]
            return (max(tgt, current_speed - self.BRAKING_RATES[1]), 'Smooth slowdown') if current_speed > tgt else (current_speed, 'Maintain')
        elif pothole_type == 2:
            tgt = self.TARGET_SPEEDS[2]
            return (max(tgt, current_speed - self.BRAKING_RATES[2]), 'Slow down / consider avoidance') if current_speed > tgt else (current_speed, 'Maintain')
        elif pothole_type == 3:
            new_spd = max(0.0, current_speed - self.BRAKING_RATES[3])
            return (0.0, 'STOPPED waiting for driver to take action!!!') if new_spd <= 0.0 else (new_spd, 'EMERGENCY BRAKING / AVOID')
```

Code Explanation: Lines 5-6 guard against actuator hunting when stationary. Lines 7-15 decrement vehicle speed at calibrated rates (2.0, 4.0, 8.0 m/s²) until the respective target ceiling (12.0, 7.0, 0.0 m/s) is achieved.